WORKSHEET — UNIT III

IoT Platforms, Edge and Cloud

Course: IoT with Applications

Unit: III (3 Hrs / 180 min)

Topics: IoT Hardware Platforms; Cloud Computing for IoT; Edge Computing; IoT + AI; Platform Comparison and Edge vs Cloud Decisions

🎯 Section A — Learning Objectives

1. Compare IoT hardware platforms — Arduino, ESP32, Raspberry Pi and NodeMCU — and explain how sensors and modules connect to them.

2. Describe IoT cloud architecture, device management, cloud data storage, cloud analytics and cloud-side dashboards.

3. Explain the need for edge computing and compare edge, fog and cloud computing on latency, bandwidth, compute power and privacy.

4. Describe real-time processing and Edge AI, and decide when processing should move from the cloud to the edge.

5. Explain how machine learning is applied to sensor data for anomaly detection, predictive maintenance and computer vision.

6. Design an end-to-end pipeline — Sensor → ESP32 → Edge AI → Cloud → Dashboard → Actuator — and select the right platform and compute location for a real-world scenario.

🔑 Section B — Key Concepts

No.	Concept	Short Explanation
1	**Microcontroller vs Single-Board Computer**	A microcontroller board (Arduino, ESP32, NodeMCU) runs one program with very low power and fast I/O control; a single-board computer (Raspberry Pi) runs a full operating system for heavier tasks.
2	**ESP32**	A low-cost 32-bit microcontroller (often dual-core) with built-in Wi-Fi and Bluetooth/BLE and deep-sleep modes, ideal for battery-powered connected nodes.
3	**Sensor Interfacing**	Sensors and modules connect to boards through analog pins (ADC), digital GPIO pins, or serial buses such as I2C, SPI and UART.
4	**IoT Cloud Architecture**	A layered cloud pipeline: device connectivity and ingestion → processing rules → storage → analytics → applications and dashboards.
5	**Device Management**	Remote registration, authentication, monitoring, configuration and over-the-air (OTA) updating of large fleets of IoT devices.
6	**Edge vs Cloud Trade-offs**	Edge gives low latency, lower bandwidth use and offline operation; the cloud gives large storage, heavy analytics and a global view of all devices.
7	**Fog Computing**	An intermediate layer of gateways or local servers between edge devices and the cloud that filters, aggregates and analyses data nearby.
8	**Edge AI**	Running a trained machine-learning model directly on a local device (inference) so decisions are instant; training usually still happens in the cloud.
9	**Anomaly Detection and Predictive Maintenance**	ML on sensor data spots abnormal behaviour and forecasts equipment failure so repairs can be planned before a breakdown.

📘 Section C — Important Terminology

Keyword	Meaning
Arduino	Open-source microcontroller platform (hardware plus IDE) known for simple C/C++ programming; the Uno uses an 8-bit ATmega328P chip.
ESP32	Espressif's low-cost 32-bit microcontroller with built-in Wi-Fi and Bluetooth/BLE, often with two cores.
Raspberry Pi	A credit-card-sized single-board computer that runs Linux and supports Python, cameras and USB devices; used as a gateway or edge node.
NodeMCU	Open-source development board built around the ESP8266 Wi-Fi chip, programmable with the Arduino IDE, Lua or MicroPython.
GPIO	General-Purpose Input/Output: programmable pins on a board that read sensors or drive actuators.
IoT Cloud Platform	Online service that connects devices and provides data ingestion, storage, analytics, dashboards and device management (e.g. AWS IoT Core, Azure IoT Hub).
Device Management	Remote registration, authentication, monitoring, configuration and updating of deployed IoT devices.
OTA Update	Over-the-air update: wireless delivery of new firmware or settings to devices already in the field.
Time-Series Database	A database optimised for storing and querying timestamped sensor readings.
Edge Computing	Processing data on or close to the device that generates it, instead of in a remote cloud data centre.
Fog Computing	A layer of gateways or local servers between the edge and the cloud that filters and aggregates data close to the source.
Edge AI	Running trained machine-learning models (inference) on edge devices; TinyML on microcontrollers is one example.
Anomaly Detection	Identifying sensor readings or patterns that deviate from normal behaviour, such as an unusual vibration spike.
Predictive Maintenance	Using sensor data and machine learning to forecast equipment failure so repairs are scheduled before a breakdown.
Computer Vision	Using cameras and machine learning to interpret images, for example detecting objects, people or product defects.

✅ Section D — Multiple-Choice Questions [Easy–Medium]

1. Which microcontroller board has built-in Wi-Fi and Bluetooth/BLE, making it popular for low-cost IoT nodes?

- **A.** Arduino Uno R3
- **B.** NodeMCU (ESP8266)
- **C.** ESP32
- **D.** A bare ATmega328P chip

2. Which platform is a single-board computer that runs a full Linux-based operating system?

- **A.** Raspberry Pi
- **B.** Arduino Uno
- **C.** ESP32
- **D.** NodeMCU

3. Which two lines make up the I2C bus used to connect many sensor modules to one board?

A. TX and RX
B. SDA and SCL
C. MOSI and MISO
D. VCC and GND

4. In an IoT cloud platform, which function registers devices, monitors their health and pushes over-the-air (OTA) firmware updates?

A. Data visualisation
B. Cloud analytics
C. Message encryption
D. Device management

5. Which type of database is best suited to storing large volumes of timestamped sensor readings?

A. Time-series database
B. Graph database
C. In-memory cache only
D. Read-only ROM

6. What is a key reason to process data at the edge instead of sending everything to the cloud?

A. Edge devices have unlimited computing power
B. Lower latency and reduced bandwidth use
C. The cloud cannot store sensor data
D. Edge computing removes the need for sensors

7. Fog computing places processing resources …

A. only inside the sensor chip
B. only in a remote data centre
C. in an intermediate layer between edge devices and the cloud, such as a local gateway
D. only on the user's mobile phone

8. In Edge AI, what usually runs on the local device?

A. Training of very large models on huge datasets
B. Long-term archiving of all raw data
C. Billing and user accounts
D. Inference using an already-trained ML model

9. Predictive maintenance uses sensor data such as vibration and temperature to …

A. estimate when equipment is likely to fail so repair can be planned in advance
B. replace machines on a fixed calendar regardless of their condition
C. reduce the number of sensors in a factory
D. increase the clock speed of a machine

10. Which technique flags sensor readings that deviate sharply from normal behaviour, such as a sudden spike in motor temperature?

A. Data compression
B. Device provisioning
C. Anomaly detection
D. Pulse-width modulation

✏️ Section E — Fill in the Blanks [Easy]

1. A _____ is a single-board computer that runs a full operating system and is often used as an IoT gateway or edge node.

2. The classic NodeMCU board is built around the _____ Wi-Fi chip.

3. Processing data close to where it is generated, instead of in a distant data centre, is called _____ computing.

4. Updating a device's firmware wirelessly, without physical access, is called an _____ update.

5. Running a trained machine-learning model directly on a local device is known as _____.

☑️ Section F — True or False [Easy]

1. The standard Arduino Uno R3 board has built-in Wi-Fi. (True / False)

2. The ESP32 supports a deep-sleep mode that helps battery-powered nodes save energy. (True / False)

3. Edge computing always replaces the cloud completely. (True / False)

4. Cloud platforms offer scalable storage and powerful analytics for IoT data. (True / False)

5. Anomaly detection can only be done in the cloud and never on an edge device. (True / False)

🔗 Section G — Match the Following [Medium]

Column A	Column B (shuffled)
1. Arduino Uno	a. Dual-core microcontroller with built-in Wi-Fi and Bluetooth
2. ESP32	b. ESP8266-based Wi-Fi development board
3. Raspberry Pi	c. Running a trained ML model on the device itself
4. NodeMCU	d. Single-board computer that runs Linux
5. Fog computing	e. 8-bit ATmega328P microcontroller board with no built-in Wi-Fi
6. Edge AI	f. Intermediate processing layer between edge devices and the cloud
7. Predictive maintenance	g. Using sensor data to predict equipment failure before it happens

📝 Section H — Short-Answer Questions [Medium]

1. Differentiate between a microcontroller board (e.g. Arduino, ESP32) and a single-board computer (e.g. Raspberry Pi).

---

2. What is device management in an IoT cloud platform? Give two tasks it covers.

---

3. Give two reasons why edge computing is needed in IoT.

---

4. What is Edge AI, and how is it different from running AI in the cloud?

---

5. What is predictive maintenance and why is it useful?

📄 Section I — Descriptive Questions [Medium–Advanced]

1. Compare Arduino, ESP32, Raspberry Pi and NodeMCU in terms of processor, connectivity, power consumption, programming and typical applications, and explain how sensors and modules connect to these boards.

---

2. Explain IoT cloud architecture — device connectivity, data storage, cloud analytics and dashboards — and describe the main functions of device management.

---

3. Compare edge, fog and cloud computing on latency, bandwidth, compute power, privacy and cost, and explain when you would choose each.

---

4. Explain how machine learning is applied to sensor data (anomaly detection, predictive maintenance, computer vision) and describe, with a diagram, the pipeline Sensor → ESP32 → Edge AI → Cloud → Dashboard → Actuator.

💻 Section J — Practical / Application-Based Questions [Advanced]

Practical 1

Problem: Write a Python program that detects an anomaly in temperature readings using the mean and standard deviation, as a simple model of anomaly detection at the edge.

Task:

1. Store the readings [28, 29, 28, 30, 29, 28, 45, 29, 30, 28] in a list.

2. Compute the mean and the standard deviation (use the statistics module).

3. Print every reading that lies more than 2 standard deviations from the mean, together with its index.

Expected Output: *Only the spike is reported, e.g. "Anomaly: 45 at index 6"; the normal readings print nothing.*

Practical 2

Problem: Choose a suitable hardware platform for each scenario and justify your choice.

Task:

1. A battery-powered soil-moisture node that reads one analog sensor and sends data over Wi-Fi every 15 minutes.

2. A door-camera unit that must recognise people locally using computer vision, with a USB camera and a full operating system.

3. A classroom robot that reads a few sensors and drives motors with tight real-time control and no internet connection.

Expected Output: *A table giving the platform, its key feature and a one-line reason for each scenario.*

Practical 3

Problem: Design the end-to-end pipeline for predictive maintenance of a factory motor: Sensor → ESP32 → Edge AI → Cloud → Dashboard → Actuator.

Task:

1. Name suitable sensors and state the ESP32's role in reading them.

2. State what the Edge AI stage does locally and what data is sent on to the cloud.

3. Describe what the cloud stores and analyses, what the dashboard shows, and what the actuator does when an anomaly is confirmed.

Expected Output: *A six-stage chain with one line per stage, e.g. vibration sensor → ESP32 → on-device anomaly model → cloud storage → dashboard alert → relay stops the motor.*

🗺️ Section K — Concept-Mapping Activity

IOT PLATFORMS, EDGE AND CLOUD

Complete the map below: fill in the missing sub-topic name or example for each branch.

Branch 1	Branch 2	Branch 3	Branch 4
Hardware Platforms	**Cloud Computing**	**Edge Computing**	**IoT + AI**
Example: _____	Example: _____	Example: _____	Example: _____

(Answer key — example values: Arduino, ESP32, Raspberry Pi, NodeMCU; device management, time-series storage, dashboards; low latency, Edge AI, fog; anomaly detection, predictive maintenance, computer vision)

🏢 Section L — Case Study / Real-World Activity

Scenario

A packaging factory runs 20 electric motors. Each motor has a vibration sensor and a temperature sensor wired to an ESP32 board. A Raspberry Pi at the end of the line collects the readings, runs a small machine-learning model to spot abnormal vibration within milliseconds, and can switch off a motor through a relay. Summarised data and alerts are sent to a cloud platform, where the maintenance manager views a dashboard.

Questions

1. Why is the ESP32 a good choice for the motor sensor nodes, and why is a Raspberry Pi suitable for the line-level device?

2. Which tasks should run at the edge and which in the cloud? Justify your answer.

3. Trace the data through the pipeline Sensor → ESP32 → Edge AI → Cloud → Dashboard → Actuator for this factory.

4. How would anomaly detection and predictive maintenance reduce downtime and repair costs?

Expected Learning Outcome

Students match hardware platforms, edge/cloud placement and AI techniques to the latency, compute and connectivity needs of a realistic IoT deployment.

📊 Section M — Self-Assessment

☐ **Yes** ☐ **Partly** ☐ **No** I can explain the major concepts of this unit.

☐ **Yes** ☐ **Partly** ☐ **No** I can define the important terminology from this unit.

☐ **Yes** ☐ **Partly** ☐ **No** I can solve problems/exercises from this unit.

☐ **Yes** ☐ **Partly** ☐ **No** I can apply the concepts of this unit to a new situation.

💭 Section N — Student Reflection

What did I learn in this unit?

What was difficult for me?

What do I need to practice more?

One important concept I learned:

🔑 Section O — Instructor Answer Key

(For instructor use — covers all objective and subjective items in this worksheet.)

MCQ Answers

- 1. C — ESP32 integrates both Wi-Fi and Bluetooth/BLE; NodeMCU (ESP8266) has Wi-Fi only and the Uno R3 has neither.
- 2. A — The Raspberry Pi is a single-board computer running Linux, unlike microcontroller boards that run a single program.
- 3. B — I2C uses two wires, SDA (data) and SCL (clock), so many modules can share one bus.
- 4. D — Device management handles registration, health monitoring and OTA firmware updates for whole fleets of devices.
- 5. A — Time-series databases are optimised for timestamped data and queries such as averages over a time window.
- 6. B — Local processing cuts response time and reduces the volume of data sent over the network.
- 7. C — Fog nodes, such as local gateways or servers, sit between edge devices and the cloud.
- 8. D — Training usually happens in the cloud; the device runs inference with the trained model.
- 9. A — Predictive maintenance forecasts failures from vibration, temperature and similar data so repairs can be planned.
- 10. C — Anomaly detection flags readings that deviate sharply from normal behaviour.

Fill in the Blanks — Answers

- 1. Raspberry Pi
- 2. ESP8266
- 3. edge
- 4. over-the-air (OTA)
- 5. Edge AI (on-device inference / TinyML)

True/False — Answers

- 1. False

- 2. True
- 3. False
- 4. True
- 5. False

Match the Following — Answer Key

1 → e 2 → a 3 → d 4 → b 5 → f 6 → c 7 → g

Short Answer — Key Points

- 1. Microcontroller boards (Arduino, ESP32) have a single chip with small memory, run one program, use very little power and excel at real-time sensor/actuator control; a single-board computer (Raspberry Pi) has a full CPU, gigabytes of RAM, Linux and USB/camera support, suiting gateways, vision and edge AI but at higher power.
- 2. It registers and authenticates devices, monitors their health and status, configures them remotely and pushes OTA firmware updates (and retires devices at end of life). Any two tasks earn full marks.
- 3. Any two: low latency for instant response; less bandwidth and lower data cost; continued operation when the internet is down; better privacy because raw data stays local.
- 4. Edge AI runs a trained model on the device itself, so decisions are instant and need no connection. Cloud AI sends data to remote servers, giving far more compute and model training but adding delay, bandwidth use and dependence on connectivity.
- 5. It uses sensor data (vibration, temperature, current) and ML to predict when equipment will fail, so maintenance is scheduled in advance — cutting unplanned downtime, repair cost and safety risk.

Descriptive Questions — Key Points

- 1. Arduino Uno: 8-bit ATmega328P at 16 MHz, no built-in Wi-Fi, very low power, simple control and learning. ESP32: 32-bit, often dual-core up to 240 MHz, Wi-Fi + Bluetooth/BLE, deep sleep, connected battery nodes. NodeMCU: ESP8266-based, Wi-Fi only, low cost, simple connected projects. Raspberry Pi: single-board computer, Linux, GBs of RAM, camera/USB, gateways, edge AI and vision. Sensors connect via analog (ADC), digital GPIO, I2C, SPI or UART. Present as a table.
- 2. Layers: device connectivity (MQTT/HTTPS) → ingestion/broker → rules and stream processing → storage (time-series database, object storage) → analytics (batch and ML) → applications (dashboards, alerts, APIs). Device management: provisioning, authentication, monitoring, configuration, OTA updates, decommissioning.
- 3. Edge: on or beside the device; millisecond latency, little bandwidth, works offline, private; limited compute and storage. Fog: gateways or local servers that aggregate and analyse nearby. Cloud: near-unlimited compute and storage, global view, model training; higher latency, bandwidth cost, needs connectivity. Choose edge for time-critical control, cloud for heavy analytics and long-term storage; hybrid designs are common.
- 4. ML workflow: collect → clean → extract features → train (cloud) → deploy (edge) → monitor. Anomaly detection flags abnormal readings (thresholds, z-score, Isolation Forest); predictive maintenance forecasts failure; computer vision uses a camera plus a neural network for detection or defect inspection. Pipeline: sensor measures → ESP32 reads and forwards → Edge AI infers locally → cloud stores, retrains and analyses → dashboard shows status and alerts → actuator (relay, valve, alarm) acts, closing the loop.

Practical Activities — Expected Outputs

- 1. Sample logic: from statistics import mean, pstdev; r = [28,29,28,30,29,28,45,29,30,28]; m, s = mean(r), pstdev(r); for i, x in enumerate(r): if abs(x - m) > 2 * s: print("Anomaly:", x, "at index", i). Mean = 30.4, standard deviation ≈ 4.92, so only 45 (about 2.97 standard deviations away) is flagged: "Anomaly: 45 at index 6".
- 2. (1) ESP32 (or NodeMCU) — built-in Wi-Fi, analog input, deep sleep for battery life. (2) Raspberry Pi — runs Linux and Python, USB camera, enough compute for vision models. (3) Arduino Uno — simple, reliable real-time I/O for sensors and motors without needing a network.
- 3. Example: vibration + temperature sensors (e.g. MPU6050 over I2C) → ESP32 reads and sends the data → Edge AI runs an anomaly model locally → only summaries and alerts go to the cloud for storage and retraining → dashboard shows trends and alerts → actuator (relay) stops the motor when an anomaly is confirmed.

Concept Map — Sample Answers

- Hardware Platforms: Arduino, ESP32, Raspberry Pi, NodeMCU. Cloud Computing: device management, time-series storage, cloud analytics, dashboards. Edge Computing: low latency, real-time processing, Edge AI, fog. IoT + AI: anomaly detection, predictive maintenance, computer vision, intelligent IoT systems.

Case Study — Key Points

- 1. ESP32: low cost, built-in Wi-Fi, reads analog and I2C sensors, low power — ideal for many small sensor nodes. Raspberry Pi: runs Linux and Python, has the compute to run an ML model, and its GPIO can drive a relay — ideal as the line-level edge node.
- 2. Edge: anomaly detection and motor shut-off, because they need millisecond response and must work even if the internet fails. Cloud: long-term storage, retraining the model on data from all motors, dashboards and alerts, and fleet management with OTA updates.
- 3. Vibration/temperature sensor → ESP32 reads it → Edge AI on the Raspberry Pi detects abnormal patterns → summaries and alerts are sent to the cloud → dashboard shows the manager the status → actuator (relay) stops the motor if needed.
- 4. Early warning of problems such as bearing wear lets repairs be planned during scheduled stops instead of after a breakdown, reducing unplanned downtime, repair cost and damage to the machine.